

Batch – T5**Assignment No. – 10****Title – Backtracking \ N Queens Problem****Student Name – Sharaneshwar Bharat Punjal****Student PRN – 23520011****Problem Statement:**

1) Given an integer n , the task is to find all distinct solutions to the n -queens problem, where n queens are placed on an $n \times n$ chessboard such that no two queens can attack each other. Each solution is a unique configuration of n queens, represented as a permutation of $[1, 2, 3, \dots, n]$. The number at the i th position indicates the row of the queen in the i th column. For example, $[3, 1, 4, 2]$ shows one such layout.

Input: 4

Output: $[2, 4, 1, 3]$, $[3, 1, 4, 2]$

Explanation: These are the 2 possible solutions.

Input: 1

Output: $[1]$

Explanation: Only one queen can be placed in the single cell available.

Input: An integer n , the size of the chessboard and the number of queens to place.Output: All distinct configurations where n queens are placed on an $n \times n$ chessboard without any attacking each other.**Code:**

```
#include <iostream>
#include <vector>
#include <cmath> // For abs()

using namespace std;

// Function to check if placing a queen at (row, col) is valid
bool is_valid(vector<int> &queens, int row, int col)
{
    for (int i = 0; i < col; i++)
    {
        // Check for row and diagonal conflicts
        if (queens[i] == row || abs(queens[i] - row) == abs(i - col))
        {
            return false;
        }
    }
    return true;
}

// Backtracking function to solve N-Queens
void place_queens(int col, vector<int> &queens, vector<vector<int>> &solutions, int n)
{
    if (col == n)
    {
        solutions.push_back(queens);
        return;
    }
    for (int row = 0; row < n; row++)
    {
        if (is_valid(queens, row, col))
        {
            queens.push_back(row);
            place_queens(col + 1, queens, solutions, n);
            queens.pop_back();
        }
    }
}
```

```
    if (col == n)
    {
        // All queens have been placed, add the solution to results
        solutions.push_back(queens); // Copy the current configuration
        return;
    }

    // Try placing a queen in every row for the current column
    for (int row = 0; row < n; row++)
    {
        if (is_valid(queens, row, col))
        {
            queens[col] = row;           // Place the queen
            place_queens(col + 1, queens, solutions, n); // Move to the next column
        }
    }
}

// Function to solve N-Queens and return all solutions
vector<vector<int>> solveNQueens(int n)
{
    vector<vector<int>> solutions;           // To store all the valid solutions
    vector<int> queens(n, -1);             // To store the position of queens
    place_queens(0, queens, solutions, n); // Start placing queens from column 0
    return solutions;
}

// Function to print the solutions in 1-based index format
void print_solutions(const vector<vector<int>> &solutions)
{
    for (const auto &sol : solutions)
    {
        for (int q : sol)
        {
            cout << q + 1 << " "; // Convert 0-based to 1-based indexing
        }
        cout << endl;
    }
}

int main()
{
    int n;
    cout << "Enter the value of N: ";
    cin >> n;

    vector<vector<int>> solutions = solveNQueens(n);
    cout << "Solutions for N = " << n << ":\n";
    print_solutions(solutions);
}
```

```
    return 0;  
}
```

Test Case 1:

Enter the value of N: 4

Solutions for N = 4:

2 4 1 3

3 1 4 2

Test Case 2:

Enter the value of N: 5

Solutions for N = 5:

1 3 5 2 4

1 4 2 5 3

2 4 1 3 5

2 5 3 1 4

3 1 4 2 5

3 5 2 4 1

4 1 3 5 2

4 2 5 3 1

5 2 4 1 3

5 3 1 4 2

Test Case 3:

Enter the value of N: 7

Solutions for N = 7:

1 3 5 7 2 4 6

1 4 7 3 6 2 5

1 5 2 6 3 7 4

1 6 4 2 7 5 3

2 4 1 7 5 3 6

2 4 6 1 3 5 7

2 5 1 4 7 3 6

2 5 3 1 7 4 6

2 5 7 4 1 3 6

2 6 3 7 4 1 5

2 7 5 3 1 6 4

3 1 6 2 5 7 4

3 1 6 4 2 7 5

3 5 7 2 4 6 1

3 6 2 5 1 4 7

3 7 2 4 6 1 5

3 7 4 1 5 2 6

4 1 3 6 2 7 5

4 1 5 2 6 3 7

4 2 7 5 3 1 6

4 6 1 3 5 7 2

4 7 3 6 2 5 1

4 7 5 2 6 1 3

5 1 4 7 3 6 2

```

5 1 6 4 2 7 3
5 2 6 3 7 4 1
5 3 1 6 4 2 7
5 7 2 4 6 1 3
5 7 2 6 3 1 4
6 1 3 5 7 2 4
6 2 5 1 4 7 3
6 3 1 4 7 5 2
6 3 5 7 1 4 2
6 3 7 4 1 5 2
6 4 2 7 5 3 1
6 4 7 1 3 5 2
7 2 4 6 1 3 5
7 3 6 2 5 1 4
7 4 1 5 2 6 3
7 5 3 1 6 4 2

```

2) Given the dimension of a chess board (N x M), determine the minimum number of queens required to cover all the squares of the board. A queen can attack any square along its row, column or diagonals.

Input : N = 8, M = 8

Output : 5

Layout :

```

Q X X X X X X X
X X Q X X X X X
X X X X Q X X X
X Q X X X X X X
X X X Q X X X X
X X X X X X X X
X X X X X X X X
X X X X X X X X

```

Input : N = 3, M = 5

Output : 2

Layout :

```

Q X X X X
X X X X X
X X X Q X

```

Code:

```

#include <iostream>
#include <vector>
using namespace std;

// Function to check if a position (row, col) is attacked by any queen
bool isAttacked(vector<vector<char>> &board, int row, int col, int N, int M)
{
    // Check the current row and column for any queen
    for (int i = 0; i < N; i++)

```

```
{
    if (board[i][col] == 'Q')
        return true;
}
for (int j = 0; j < M; j++)
{
    if (board[row][j] == 'Q')
        return true;
}

// Check diagonals
for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
{
    if (board[i][j] == 'Q')
        return true;
}
for (int i = row, j = col; i >= 0 && j < M; i--, j++)
{
    if (board[i][j] == 'Q')
        return true;
}
for (int i = row, j = col; i < N && j >= 0; i++, j--)
{
    if (board[i][j] == 'Q')
        return true;
}
for (int i = row, j = col; i < N && j < M; i++, j++)
{
    if (board[i][j] == 'Q')
        return true;
}

return false;
}

// Function to place the minimum number of queens on the board
void placeQueens(vector<vector<char>> &board, int N, int M)
{
    int queensPlaced = 0;

    // Try placing queens in an alternating pattern
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < M; j++)
        {
            if (!isAttacked(board, i, j, N, M))
            {
                board[i][j] = 'Q';
                queensPlaced++;
            }
        }
    }
}
```

```
    }  
}  
  
// Output the board  
cout << "Minimum number of queens required: " << queensPlaced << endl;  
for (int i = 0; i < N; i++)  
{  
    for (int j = 0; j < M; j++)  
    {  
        cout << board[i][j] << " ";  
    }  
    cout << endl;  
}  
}  
  
int main()  
{  
    int N, M;  
    cout << "Enter dimensions of the chessboard (N M): ";  
    cin >> N >> M;  
  
    vector<vector<char>> board(N, vector<char>(M, 'X'));  
  
    // Place queens and print the layout  
    placeQueens(board, N, M);  
  
    return 0;  
}
```

Test Case 1

Enter dimensions of the chessboard (N M): 8 8

Minimum number of queens required: 5

```
Q X X X X X X X  
X X Q X X X X X  
X X X X Q X X X  
X Q X X X X X X  
X X X Q X X X X  
X X X X X X X X  
X X X X X X X X  
X X X X X X X X
```

Test Case 2

Enter dimensions of the chessboard (N M): 5 5

Minimum number of queens required: 5

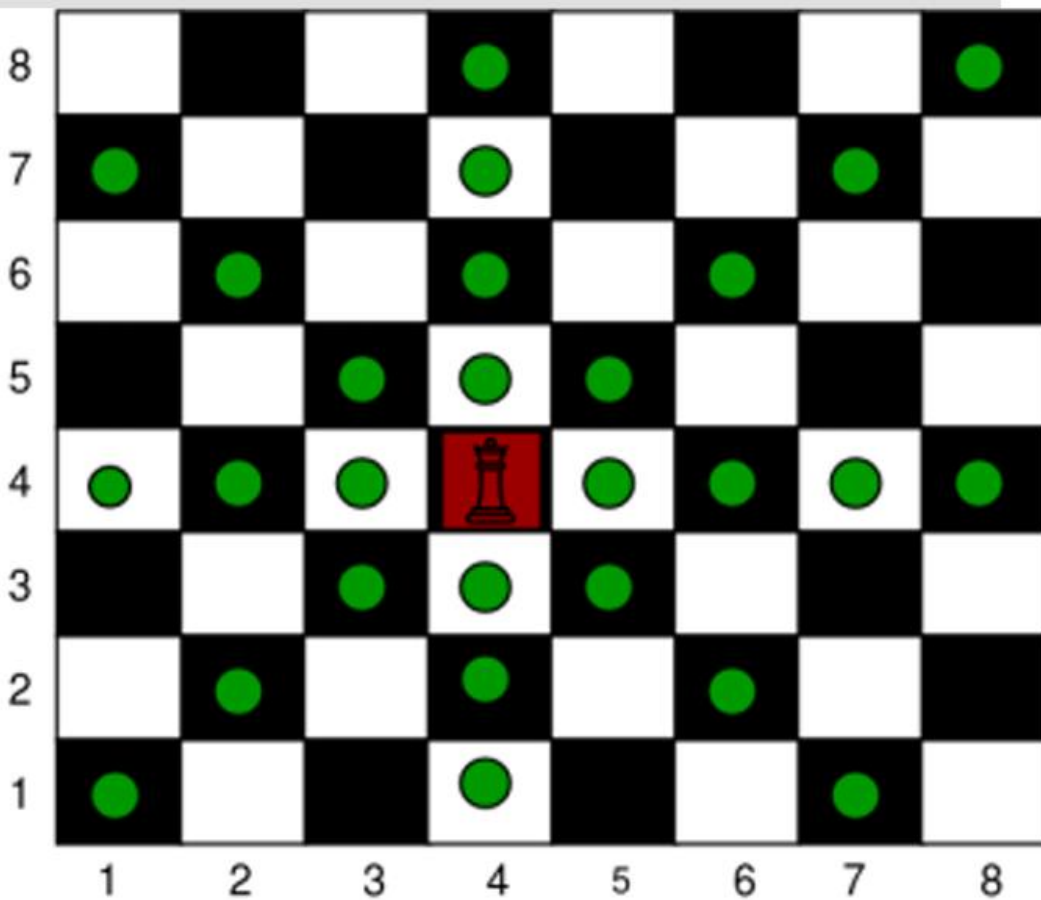
```
Q X X X X  
X X Q X X  
X X X X Q  
X Q X X X  
X X X Q X
```

3) Consider a N X N chessboard with a Queen and K obstacles. The Queen cannot pass through obstacles. Given the position (x, y) of Queen, the task is to find the number of cells the queen can move.

Input : N = 8, x = 4, y = 4,

K = 0

Output : 27



Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// Function to calculate the number of cells the Queen can move
int countQueenMoves(int N, int Qx, int Qy, vector<pair<int, int>> &obstacles)
{
    // Directions: up, down, left, right, top-left, top-right, bottom-left, bottom-
    right
    int up = N - Qx;
    int down = Qx - 1;
```

```
int left = Qy - 1;
int right = N - Qy;
int topLeft = min(N - Qx, Qy - 1);
int topRight = min(N - Qx, N - Qy);
int bottomLeft = min(Qx - 1, Qy - 1);
int bottomRight = min(Qx - 1, N - Qy);

// Adjust boundaries based on obstacles
for (auto &obs : obstacles)
{
    int ox = obs.first, oy = obs.second;

    // Same column (vertical movement)
    if (oy == Qy)
    {
        if (ox > Qx)
        {
            up = min(up, ox - Qx - 1); // obstacle above
        }
        else
        {
            down = min(down, Qx - ox - 1); // obstacle below
        }
    }

    // Same row (horizontal movement)
    if (ox == Qx)
    {
        if (oy > Qy)
        {
            right = min(right, oy - Qy - 1); // obstacle to the right
        }
        else
        {
            left = min(left, Qy - oy - 1); // obstacle to the left
        }
    }

    // Diagonal movements
    if (abs(ox - Qx) == abs(oy - Qy))
    {
        if (ox > Qx && oy > Qy)
        {
            topRight = min(topRight, ox - Qx - 1); // obstacle top-right
        }
        else if (ox > Qx && oy < Qy)
        {
            topLeft = min(topLeft, ox - Qx - 1); // obstacle top-left
        }
        else if (ox < Qx && oy > Qy)
```

```
        {
            bottomRight = min(bottomRight, Qx - ox - 1); // obstacle bottom-right
        }
        else if (ox < Qx && oy < Qy)
        {
            bottomLeft = min(bottomLeft, Qx - ox - 1); // obstacle bottom-left
        }
    }
}

// Total possible moves
return up + down + left + right + topLeft + topRight + bottomLeft + bottomRight;
}

int main()
{
    int N, K, Qx, Qy;

    // Inputs
    cout << "Enter the size of the board (N): ";
    cin >> N;

    cout << "Enter the Queen's position (Qx, Qy): ";
    cin >> Qx >> Qy;

    cout << "Enter the number of obstacles (K): ";
    cin >> K;

    vector<pair<int, int>> obstacles(K);
    if (K > 0)
    {
        cout << "Enter the positions of obstacles (ox oy): " << endl;
        for (int i = 0; i < K; i++)
        {
            cin >> obstacles[i].first >> obstacles[i].second;
        }
    }

    int result = countQueenMoves(N, Qx, Qy, obstacles);
    cout << "The number of cells the Queen can move to: " << result << endl;

    return 0;
}
```

Test Case 1

```
Enter the size of the board (N): 8
Enter the Queen's position (Qx, Qy): 4 4
Enter the number of obstacles (K): 2
Enter the positions of obstacles (ox oy):
5 5
```

3 2

The number of cells the Queen can move to: 23

Test Case 2

Enter the size of the board (N): 8

Enter the Queen's position (Qx, Qy): 1 1

Enter the number of obstacles (K): 0

The number of cells the Queen can move to: 21

Test Case 3

Enter the size of the board (N): 8

Enter the Queen's position (Qx, Qy): 4 4

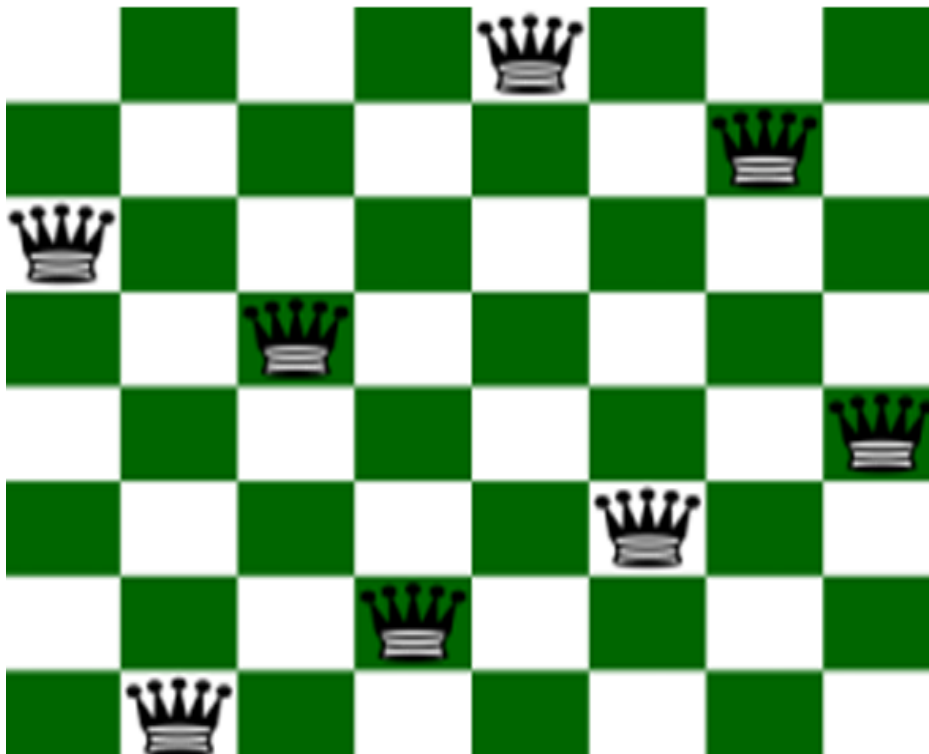
Enter the number of obstacles (K): 1

Enter the positions of obstacles (ox oy):

6 4

The number of cells the Queen can move to: 24

4) The N Queen is the problem of placing N chess queens on an N×N chessboard so that no two queens attack each other. For example, the following is a solution for 8 Queen problem.



Code:

```
#include <iostream>
#include <vector>
using namespace std;

class NQueens
{
private:
    int N;
    vector<vector<string>> solutions;

    // Function to check if placing a queen at (row, col) is safe
    bool isSafe(vector<string> &board, int row, int col)
    {
        // Check this row on left side
        for (int i = 0; i < col; i++)
        {
            if (board[row][i] == 'Q')
                return false;
        }

        // Check upper diagonal on left side
        for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
        {
            if (board[i][j] == 'Q')
                return false;
        }

        // Check lower diagonal on left side
        for (int i = row, j = col; j >= 0 && i < N; i++, j--)
        {
            if (board[i][j] == 'Q')
                return false;
        }

        return true;
    }

    // Helper function to solve the problem using backtracking
    void solveNQueensUtil(vector<string> &board, int col)
    {
        // If all queens are placed
        if (col == N)
        {
            solutions.push_back(board);
            return;
        }

        // Try placing a queen in each row of this column
        for (int i = 0; i < N; i++)
```

```
        {
            if (isSafe(board, i, col))
            {
                board[i][col] = 'Q';           // Place the queen
                solveNQueensUtil(board, col + 1); // Recur for next column
                board[i][col] = '.';           // Backtrack and remove the queen
            }
        }
    }

public:
    // Constructor to initialize board size
    NQueens(int n) : N(n) {}

    // Function to solve the N-Queens problem
    vector<vector<string>> solveNQueens()
    {
        vector<string> board(N, string(N, '.'));
        solveNQueensUtil(board, 0);
        return solutions;
    }
};

int main()
{
    int N;
    cout << "Enter the value of N: ";
    cin >> N;

    NQueens solver(N);
    vector<vector<string>> result = solver.solveNQueens();

    if (result.empty())
    {
        cout << "No solution exists for N = " << N << endl;
    }
    else
    {
        cout << "Number of solutions: " << result.size() << endl;
        for (const auto &solution : result)
        {
            for (const string &row : solution)
            {
                cout << row << endl;
            }
            cout << endl;
        }
    }

    return 0;
}
```

```
}
```

Test Case 1

Enter the value of N: 4

Number of solutions: 2

..Q.

Q...

...Q

.Q..

.Q..

...Q

Q...

..Q.

Test Case 2

Enter the value of N: 1

Number of solutions: 1

Q

Test Case 3

Enter the value of N: 2

No solution exists for N = 2

Test Case 4

Enter the value of N: 5

Number of solutions: 10

5) Given a valid sentence without any spaces between the words and a dictionary of valid English words, find all possible ways to break the sentence into individual dictionary words.

Example:

Consider the following dictionary

```
{ i, like, sam, sung, samsung, mobile, ice,  
  and, cream, icecream, man, go, mango }
```

Input: "ilikesamsungmobile"

Output: i like sam sung mobile

i like samsung mobile

Input: "ilikeicecreamandmango"

Output: i like ice cream and man go

i like ice cream and mango

i like icecream and man go

i like icecream and mango

Code:

```
#include <iostream>
#include <unordered_set>
#include <vector>
using namespace std;

// Function to check if a word is present in the dictionary
bool dictionaryContains(string word, unordered_set<string> &dictionary)
{
    return dictionary.find(word) != dictionary.end();
}

// Helper function to perform backtracking and find all word breaks
void wordBreakUtil(string s, unordered_set<string> &dictionary, string result,
vector<string> &output)
{
    // If we've reached the end of the string, add the result to the output
    if (s.size() == 0)
    {
        output.push_back(result);
        return;
    }

    // Try every prefix of the string
    for (int i = 1; i <= s.size(); i++)
    {
        string prefix = s.substr(0, i);

        // If the prefix is a valid word, proceed to check the remaining substring
        if (dictionaryContains(prefix, dictionary))
        {
            string newResult = result + (result.empty() ? "" : " ") + prefix;
            wordBreakUtil(s.substr(i), dictionary, newResult, output);
        }
    }
}

// Main function to find all word breaks
vector<string> wordBreak(string s, unordered_set<string> &dictionary)
{
    vector<string> output;
    wordBreakUtil(s, dictionary, "", output);
    return output;
}

int main()
{
    // Create a dictionary of valid words
    unordered_set<string> dictionary = {"i", "like", "sam", "sung", "samsung",
"mobile", "ice", "and", "cream", "icecream", "man", "go", "mango"};
```

```
// Input sentence without spaces
vector<string> inputs = {
    "ilikesamsungmobile",
    "likeicecreamandmango",
    "samsungandmango",
    "icecreamandmango"};

// Iterate over the inputs and get possible word breaks for each
for (const string &input : inputs)
{
    vector<string> result = wordBreak(input, dictionary);
    cout << "Possible segmentations for input \"" << input << "\":" << endl;
    for (const string &sentence : result)
    {
        cout << sentence << endl;
    }
    cout << endl;
}

return 0;
}
```

Test Case 1

Possible segmentations for input "ilikesamsungmobile":

i like sam sung mobile

i like samsung mobile

Test Case 2

Possible segmentations for input "likeicecreamandmango":

i like ice cream and man go

i like ice cream and mango

i like icecream and man go

i like icecream and mango

Test Case 3

Possible segmentations for input "samsungandmango":

sam sung and man go

sam sung and mango

samsung and man go

samsung and mango

Test Case 4

Possible segmentations for input "icecreamandmango":

ice cream and man go

ice cream and mango

icecream and man go

icecream and mango